ISLAMIC CALL UNIVERSITY COLLEGE.

FACULTY OF SCIENCE AND TECHNOLOGY.

MOBILE APPLICATION.

PROJECT: A SCHOOL LIBRARY MANAGEMENT SYSTEM.

NANKUMBA SALMAH 241- 22302- 7201

NAKABUYE HAFSWA 241- 22302- 7171

LUKSA HARUNAH 241-22302-7421

INTRODUCTION.

The school library management system is a mobile application designed to help a school manage its library books and borrowing activities. The system will allow authorised users to log in, view available books, and manage borrowing records. Library administrators will be able to add, view, update and delete book and category records.

The application will use flutter for the mobile interface, fast API for the back end REST API, and MYSQL for storing data.

PROBLEM STATEMENT.

Some school libraries manage book information and borrowing activities using paper records or spreadsheets. These methods can make it difficult to find books quickly, keep borrowing records accurate, track returned books and update information when changes occur.
The proposed school library management system will provide a centralized digital platform for managing books, categories and borrowing records. It aims improve access to library information, reduce record keeping errors and make library operations easier.

OBJECTIVES.

The general objective is to design and develop a mobile bases school library management system that improves the management of library books and borrowing activities.
To develop a secure registration and login system.
To create a database for storing users, books categories and borrowing records.
To allow authorised administrators to manage book categories.
To allow authorised administrators to manage book categories.
To enable users to view available books.
To protect user information through authentication and access control
to test the system functionality and API endpoints.

SYSTEM ANALYSIS.

TARGET USERS.

Administrators: manages book information, categories and borrowing records, and oversees library activities.

Students; logs in, searches or view available books, and views their own borrowing records. The ability to borrow or request a book through the app will be subject to the final borrowing workflow.

EXISTING SYSTEM.

The existing library process may involve paper bases record or spreadsheets. Such methods can require manual searching and updating and may make it harder to track book availability and borrowing history.

PROPOSED SYSTEM

The proposed system will provide a flutter mobile application connected to a FastAPI backend and MySQL database. Users will interact with the mobile application, while the backend will validate requests, enforce permissions and communicate with the database.

SCOOP OF THE SYSTEM.

The system will cover;

user registration, login and logout.
Secure access to protected features.
Book and category management
viewing available books.
Recording borrowing and returning activities
viewing borrowing history and status
validating user input and displaying errors
storing and retrieving information from MySQL

The system will not initially include online payment gateways or integration with external library systems.

SYSTEM REQUIREMENTS.

FUNCTIONAL REQUIREMENTS.

The system shall;

1. allow a new user to register an account.
2. allow registered users to log in using valid credentials.
3. allow authenticated users to log out by clearing their authentication token.
4. allow authorised administrators to update book information.
5. allow users to view the list of books and individual book details.
6. allow authorised administrators to delete book records when permitted
7. allow authorised administrators to create, view update and delete book categories
8. allow users to view their own borrowing record.
9. allow the system to validate input and return appropriate error messages.
10. prevent ordinary users from performing administrator only actions.

NON -FUNCTIONAL REQUIREMENTS.

Security: passwords must be hashed using b crypt or an appropriate password-hashing implementation. Protected endpoints must require a valid authentication token.

Usability: the application must have a simple clear interface that users can navigate easily.
Reliability: the back end must validate requests and handle errors without crashing
performance: the application should return ordinary book and borrowing queries within a reasonable response time under normal testing conditions.
Maintainability: back end, database, models, services and flutter screens should be organized into separate files and folders.

USER STORIES

1. as a new user, I want to register an account so that I can access the application.
2. as a registered user, I want to log in securely so that I can access authorised features.
3. as a user, I want to view available books so that I can find books of interest
4. as an administrator, I want to add books so that new library items can be recorded
5. as an administrator, I want to update book information so that records remain accurate.

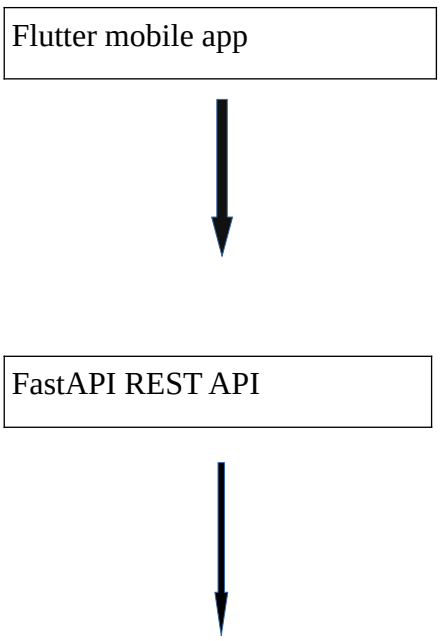
6. as an administrator, I want to delete permitted book records so that outdated records can be removed.
7. as an administrator, I want to organise books into categories so that books are easier to find.
8. as a library user, I want to view my borrowing records so that I can track borrowed books.
9. as an authorised user, I want to record a book return so that its borrowing status can be updated.
10. as a user, I want to receive clear error messages so that I can correct invalid information.

SYSTEM ARCHITECTURE

the system will use a three-layer architecture.
1. presentation layer: flutter and dart provide the mobile screens, forms, naigation, loading indicators and error messages
2. application layer: python fastAPI exposes REST API endpoints, validates requests, authenticates users and applies business rules.
3. data layer: MySQL stores user accounts, books categories and borrowing records.

The mobile app will send HTTP requests to the backend. The backend will process the requests and return JSON responses.

ARCHITECTURE FLOW



MySQL database

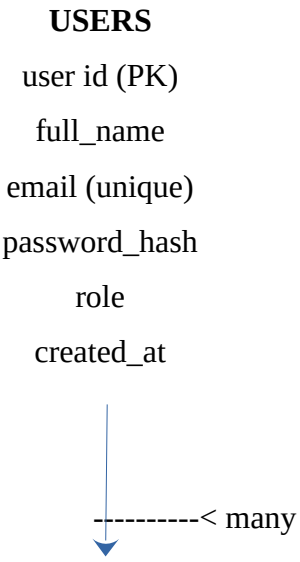
The database will not be accesses directly by the mobile app.

DATABASE DESIGN

the database will be named school_libary_db
it will contain for main tables users, categories, books and borrowing.

ENTITY RELATIONSHIP DIAGRAM (ERD)

the following is the planned relationship structure.



status



many>---------1

BOOKS

book_id (PK)

category_id (FK)

title

author

isbn (UNIQUE)

quality

available_quality

created_at



many>-----1

CATEGORIES

category_id (PK)

category_name (UNIQUE)

description

RELATIONSHIPS

- one user can have many borrowing records.
- One book can appear in many borrowing records over time
- one category can contain many books
- each borrowing record belongs to one user and one book
- each book belongs to one category.

PK means primary key, a field that uniquely identifies a record. FK means foreign key, a field that links a record to another table.

DATA DICTIONARY.

USER TABLE

FIELD	DATA TYPE	DESCRIPTION
user_id	INT, PK, AUTO_INCREMENT	Unique user identifier
full_name	VARCHAR(100)	Users full name
email	VARCHAR(150)	Login email
password_hash	VARCHAR(255)	Hashed password
role	VARCHAR(20)	Administrator or student
created_at	TIMESTAMP	Account creation date

CATEGORIES TABLE.

FIELD	DATA TYPE	DESCRIPTION
category_id	INT, PK, AUTO INCREMENT	Unique category identifier
category_name	VARCHAR(100) UNIQUE	Name of category
description	VARCHAR(255)	Category description

BOOKS TABLE.

FIELD	DATATYPE	DESCRIPTION
Book_id	INT, PK AUTO INCREMENT	Unique booki identifier
category_id	INT, PK	References categories, category-id
title	VARCHAR(100)	Book title
author	VARCHAR(100)	Authors name

isbn	VARCHAR(30) UNIQUE	Book isbn
quantity	INT	Total copies owned
available_quality	INT	Copies currently available
created_at	TIMESTAMP	Record creation date

BORROWING TABLE

FIELD	DATA TYPE	DESCRIPTION
borrowing_id	INT, PK AUTO INCREMENT	Unique book identifier
user_id	INT, PK	Reference users. user_id
book_id	INT, PK	References books. book_id
borrow_date	DATE	Date borrowed
due_date	DATE	Expected return date
return_date	DATE, NULL	Actual return date
status	VARCHAR(20)	Borrowed or returned

The backend will enforce valid quantities, unique email addresses and ISBNs, Valid foreign key references.

API ENDPOINT DESIGN.

The backend will use REST endpoints and JSON for request and response data. The following table describes the planned endpoints.

Method	**Endpoint**	**Purpose**	**Access**
POST	*/auth/register*	Register a new user	public
POST	*/auth/login*	Authenticate user and return token	public
POST	*/auth/login*	Log out and invalidate/clear token	authenticated

		as implemented	
GET	*/books*	List of books	authenticated
GET	*/books/{book_id*	Get one book	authenticated
POST	*/books*	Create a book	admin
PUT	*/books/{book_id}*	Update a date	admin
DELETE	*/books/{book_id}*	Delete a permitted book	admin
GET	*/categories*	List categories	authenticated
GET	*category/{category_id}*	Get one category	authenticated
POST	*/categories/*	Create a category	admin
PUT	*category/{category_id}*	Update a category	admin
DELETE	*category/{category_id}*	Delete a permitted category	admin
POST	*/borrowings*	Create a borrowing record	Authenticated, subject to rules
GET	*/borrowings*	List the current users records admin may have broader access	authenticated
GET	*/borrowing/ {borrowing_id}*	View an authorised borrowing record	Owner or admin
PUT	*/borrowing/ {borrowing_id}*	Update permitted borrowing details or return status	Owner or admin subject rules
DELETE	*S/borrowing/ {borrowing_id}*	Delete or permitted record if allowed by policy	Sadmin

EXPECTED HTTP STATUS CODES.

200 OK: request succeeded

201 created: a record was created successfully.
204 No Content: A deletion or other operation succeeded without a response body, where used.
400 bad request: request cannot be processed because of invalid input
401 Unauthorized: authentication is missing or invalid
403 forbidden: the user is authenticated but lacks permission.
404 conflict: a duplicate or other data conflict occurred
422 Unprocessable entity: Fast API validation rejected the request.

The implementation will use consistent error responses and document the actual responses in the API documentation.

AUTHENTICATION AND AUTHORIZATION.

Passwords will never be stored as plain text. After successful login, the backend will issue an authentication token. Protected endpoints will validate the token before processing requests.

Administrators will be allowed to manage books and categories. Ordinary users will not be allowed to use administrator-only endpoints. Users must only access their own borrowing records unless they have authorised administrative privileges.

The logout implementation must invalidate the token server-side or otherwise ensure it can no longer be used. if token invalidation is supported by the chosen authentication approach.

MOBILE APPLICATION SCREEN DESIGN

The flutter application will contain at least the following screens:

1. login screen: email and password fields, login button and link to registration.
2. registration screen: full name, email , password and account creation button
3. dashboard screen: navigation to books, categories and borrowing records according to the users role
4. books screen: displays books, availability and a way to view book details
5. book form screen: allows administrators to add or edit book details.
6. categories screen: displays and manages book categories

7. borrowing screen; displays borrowing history and relevant return status., authorised users can record or update borrowing activities.

Each screen will include appropriate loading states, input validation and error messages. The interface will communicate with the FastAPI backend rather than relying on hard coded sample data.

Simple wire frame sketches for the login, dashboard, books and borrowing screens will be added to the final document.

TESTING PLAN

the following tests will be performed during development;

- register with valid details and reject invalid or duplicate details.
- Login with correct credentials and reject incorrect credentials
- reject protected API requests without a valid token
- confirm that ordinary users cannot perform administrator-only actions.
- Create, view update and delete book records.
- Create ,view, update, and delete category records.
- Check that foreign keys and duplicate constraints work correctly
- check borrowings, book availability and return-status changes
- confirm that users cannot view another users private borrowing records.
- Test the API using postman
- test that the flutter app displays data and handles loading and errors correctly

DEVELOPMENT TOOLS.

Flutter and dart: mobile application
python and FastAPI: backend REST API
MySQL: relational database
Visual studio code: code editing and development.
Postman: API testing
Git and GitHub: version control and collaborations

EXPECTED OUTCOME.

The expected outcome is a working mobile application connected to a MySQL database through a FastAPI backend. The system will support secure user authentication, book and category management, and borrowing record management. It will provide a simple interface and enforce appropriate access permissions and data validation.

CONCLUSION.

The proposed school library management system aims to make school library operations easier to manage by providing a centralized mobile platform for book information and borrowing records. The design described in this document will guide the implementation, testing and intergration of the backend and flutter application.